De app de Clima a app de Pokémon

Paso a paso para cambiar tu proyecto (`com.example.clima`) de **WeatherAPI** a **PokeAPI** (`https://pokeapi.co/api/v2/pokemon/ditto`).

Antes de empezar: qué cambia y qué no

PokeAPI es **pública y gratis**: **no necesita API key**. Por eso desaparece todo lo relacionado con la clave.

Como pasas de datos de **clima** a datos de **Pokémon**, ya no basta con cambiar la capa `datos/`. También cambian las **entidades**, el **ViewModel** y la **UI**. Lo que sí se mantiene es **la arquitectura**:

```
Internet → ApiClient → PokeApiService → PokemonDto → Mapper → Pokemon (entidad)
          → PokemonRepository → PokemonViewModel → PokemonApp (UI)
```

Mapa de archivos

Archivo actual	Qué hacer	Archivo nuevo
`apikey.properties(.example)`	Borrar	—
`app/build.gradle.kts`	Quitar las líneas de la clave	(el mismo)
`datos/ApiClient.kt`	Cambiar URL base y tipo	(el mismo)
`datos/WeatherApiService.kt`	Borrar	`datos/PokeApiService.kt`
`datos/dto/ClimaDtos.kt`	Borrar	`datos/dto/PokemonDtos.kt`
`datos/mapper/ClimaMapper.kt`	Borrar	`datos/mapper/PokemonMapper.kt`
`datos/ClimaRepository.kt`	Borrar	`datos/PokemonRepository.kt`
`datos/DatosDemo.kt`	Borrar	— (ya no hace falta modo demo)
`entidad/` (los 7 archivos de clima)	Borrar	`entidad/Pokemon.kt`
`presentacion/ClimaViewModel.kt`	Borrar	`presentacion/PokemonViewModel.kt`
`ui/ClimaApp.kt`	Borrar	`ui/PokemonApp.kt`
`ui/ClimaTheme.kt`	Se queda igual	(el mismo)
`MainActivity.kt`	Usar el ViewModel y la UI nuevos	(el mismo)
`test/ClimaRepositoryTest.kt` + `clima.json`	Borrar	`PokemonRepositoryTest.kt` + `ditto.json`

> **Importante**
> El proyecto **no compilará** hasta que termines todos los pasos, porque los archivos dependen unos de otros. Es normal ver errores en rojo mientras avanzas. Compila al final, en el Paso 11.

Cómo crear un archivo nuevo en Android Studio
Clic derecho en la carpeta (por ejemplo `datos`) → **New** → **Kotlin Class/File** → escribe el nombre → elige **File**. Después pega el código del paso.

Paso 0 · Preparación

- ☐ Haz una **copia de seguridad** de la carpeta del proyecto (cópiala y pégala con otro nombre).
- ☐ Abre `https://pokeapi.co/api/v2/pokemon/ditto` en el navegador. Vas a ver un JSON muy largo.
- ☐ Guárdalo: **Ctrl + S** → nombre `ditto.json`. Lo usarás en los tests (Paso 10).
- ☐ Fíjate en estos campos del JSON. Son los que usará la app:

Campo en el JSON	Ejemplo (Ditto)	Ojo
`id`	132	Número de Pokédex
`name`	"ditto"	Siempre en minúsculas
`height`	3	En **decímetros**: ÷ 10 = 0.3 m
`weight`	40	En **hectogramos**: ÷ 10 = 4.0 kg
`sprites.front_default`	"https://raw.githubusercontent.com/.../132.png"	URL de la imagen (puede ser `null`)
`types[].type.name`	"normal"	Lista: un Pokémon puede tener 2 tipos
`abilities[].ability.name`	"limber", "imposter"	Lista
`stats[].base_stat` / `stats[].stat.name`	48 / "hp"	Lista de 6 estadísticas

Paso 1 · Quitar la API key

- ☐ Borra `apikey.properties` (si existe) y `apikey.properties.example` de la raíz del proyecto.
- ☐ En `app/build.gradle.kts` borra **las líneas 8 a 14** (desde `val secrets = …` hasta `val escapedKey = …`).
- ☐ En el mismo archivo borra **la línea 25**: `buildConfigField("String", "WEATHER_API_KEY", …)`.
- ☐ Si `import java.util.Properties` (línea 1) queda sin usar, también puedes borrarlo.
- ☐ Pulsa **Sync Project with Gradle Files**.

Paso 2 · Cambiar la URL base

`datos/ApiClient.kt`

```
package com.example.clima.datos

import okhttp3.OkHttpClient
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit
```

```
object ApiClient {
    val api: PokeApiService by lazy {
        val client = OkHttpClient.Builder()
            .connectTimeout(15, TimeUnit.SECONDS)
            .readTimeout(20, TimeUnit.SECONDS)
            .callTimeout(30, TimeUnit.SECONDS)
            .build()
        Retrofit.Builder()
            .baseUrl("https://pokeapi.co/api/v2/")      // ← debe terminar en "/"
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(PokeApiService::class.java)
    }
}
```

Paso 3 · El endpoint

Borra `WeatherApiService.kt` y crea `PokeApiService.kt`. El nombre del Pokémon va **dentro de la ruta**, así que se usa `@Path` en lugar de `@Query`:

`datos/PokeApiService.kt`

```
package com.example.clima.datos

import com.example.clima.datos.dto.PokemonDto
import retrofit2.http.GET
import retrofit2.http.Path

interface PokeApiService {
    // {nombre} se reemplaza: pokemon/ditto, pokemon/pikachu, pokemon/25…
    @GET("pokemon/{nombre}")
    suspend fun obtenerPokemon(@Path("nombre") nombre: String): PokemonDto
}
```

Paso 4 · Los DTOs

Borra `ClimaDtos.kt` y crea `PokemonDtos.kt`. Solo hace falta declarar los campos que vas a usar. Gson ignora el resto del JSON.

`datos/dto/PokemonDtos.kt`

```
package com.example.clima.datos.dto

import com.google.gson.annotations.SerializedName

// Modelos de los datos que llegan desde PokeAPI.
data class PokemonDto(
    @SerializedName("id") val id: Int,
    @SerializedName("name") val nombre: String,
    @SerializedName("height") val altura: Int,          // decímetros
    @SerializedName("weight") val peso: Int,            // hectogramos
    @SerializedName("sprites") val sprites: SpritesDto,
    @SerializedName("types") val tipos: List<TipoSlotDto>,
    @SerializedName("abilities") val habilidades: List<HabilidadSlotDto>,
    @SerializedName("stats") val estadisticas: List<EstadisticaDto>
)

data class SpritesDto(
    @SerializedName("front_default") val frente: String?   // puede venir null
)

data class TipoSlotDto(@SerializedName("type") val tipo: RecursoDto)

data class HabilidadSlotDto(@SerializedName("ability") val habilidad: RecursoDto)

data class EstadisticaDto(
    @SerializedName("base_stat") val valor: Int,
    @SerializedName("stat") val stat: RecursoDto
)

// PokeAPI repite mucho este formato: { "name": "...", "url": "..." }
data class RecursoDto(@SerializedName("name") val nombre: String)
```

> **Cómo leer el anidamiento**
> En el JSON, `"types": [ { "slot": 1, "type": { "name": "normal" } } ]` es una **lista** de objetos, y cada uno tiene dentro otro objeto `type`. Por eso hay tres niveles: `List<TipoSlotDto>` → `tipo: RecursoDto` → `nombre`.

Paso 5 · La entidad nueva

Borra los 7 archivos de `entidad/` (`ClimaActual`, `Condicion`, `DiaPronostico`, `Pronostico`, `RespuestaClima`, `ResumenDia`, `Ubicacion`) y crea:

`entidad/Pokemon.kt`

```
package com.example.clima.entidad

// Lo que la app necesita saber de un Pokémon, ya en unidades normales.
data class Pokemon(
```

```
    val id: Int,
    val nombre: String,
    val alturaM: Double,
    val pesoKg: Double,
    val imagen: String,
    val tipos: List<String>,
    val habilidades: List<String>,
    val estadisticas: List<Estadistica>
)

data class Estadistica(val nombre: String, val valor: Int)
```

Paso 6 · El mapper

Borra `ClimaMapper.kt` y crea `PokemonMapper.kt`. Aquí se convierten las unidades y se "aplanan" las listas anidadas:

`datos/mapper/PokemonMapper.kt`

```
package com.example.clima.datos.mapper

import com.example.clima.datos.dto.PokemonDto
import com.example.clima.entidad.Estadistica
import com.example.clima.entidad.Pokemon

// Transforma el DTO recibido de PokeAPI en la entidad de la app.
fun PokemonDto.aEntidad(): Pokemon = Pokemon(
    id = id,
    nombre = nombre.replaceFirstChar { it.uppercase() },   // "ditto" → "Ditto"
    alturaM = altura / 10.0,                                // decímetros → metros
    pesoKg = peso / 10.0,                                   // hectogramos → kg
    imagen = sprites.frente.orEmpty(),
    tipos = tipos.map { it.tipo.nombre },
    habilidades = habilidades.map { it.habilidad.nombre },
    estadisticas = estadisticas.map { Estadistica(it.stat.nombre, it.valor) }
)
```

Paso 7 · El repositorio

Borra `ClimaRepository.kt` y `DatosDemo.kt`, y crea `PokemonRepository.kt`. Ya no hay clave que comprobar. Si el Pokémon no existe, PokeAPI responde con **HTTP 404**.

`datos/PokemonRepository.kt`

```
package com.example.clima.datos

import com.example.clima.datos.mapper.aEntidad
import com.example.clima.entidad.Pokemon
import com.google.gson.JsonParseException
import kotlinx.coroutines.CancellationException
import retrofit2.HttpException
import java.io.IOException
import java.net.SocketTimeoutException

class PokemonException(message: String) : Exception(message)

class PokemonRepository(private val api: PokeApiService) {
    suspend fun obtenerPokemon(nombre: String): Pokemon {
        // PokeAPI solo acepta minúsculas y guiones: "Mr Mime" → "mr-mime"
        val consulta = nombre.trim().lowercase().replace(' ', '-')
        if (consulta.isEmpty()) throw PokemonException("Escribe el nombre o número de un Pokémon.")
        try {
            return api.obtenerPokemon(consulta).aEntidad()
        } catch (e: CancellationException) {
            throw e
        } catch (e: HttpException) {
            throw PokemonException(
                if (e.code() == 404) "No encontramos ese Pokémon. Revisa cómo está escrito."
                else "El servicio no pudo responder (HTTP ${e.code()}). Intenta nuevamente."
            )
        } catch (e: SocketTimeoutException) {
            throw PokemonException("La consulta tardó demasiado. Intenta nuevamente.")
        } catch (e: IOException) {
            throw PokemonException("No se pudo conectar. Comprueba tu conexión a Internet.")
        } catch (e: JsonParseException) {
            throw PokemonException("El servicio devolvió datos que no se pudieron interpretar.")
        }
    }
}
```

Paso 8 · El ViewModel

Borra `ClimaViewModel.kt` y crea `PokemonViewModel.kt`. Es más simple que el anterior porque ya no hay modo demo:

`presentacion/PokemonViewModel.kt`

```
package com.example.clima.presentacion

import androidx.lifecycle.SavedStateHandle
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.clima.datos.*
import com.example.clima.entidad.Pokemon
import kotlinx.coroutines.CancellationException
```

```
import kotlinx.coroutines.Job
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

data class PokemonUiState(
    val busqueda: String = "ditto",
    val cargando: Boolean = false,
    val error: String? = null,
    val pokemon: Pokemon? = null
)

class PokemonViewModel(private val savedState: SavedStateHandle) : ViewModel() {
    private val repository = PokemonRepository(ApiClient.api)
    private val _estado = MutableStateFlow(PokemonUiState(busqueda = savedState["busqueda"] ?:
"ditto"))
    val estado = _estado.asStateFlow()
    private var consulta: Job? = null

    init { buscar() }

    fun cambiarBusqueda(valor: String) {
        savedState["busqueda"] = valor
        _estado.update { it.copy(busqueda = valor) }
    }

    fun buscar(texto: String = _estado.value.busqueda) {
        consulta?.cancel()
        _estado.update { it.copy(cargando = true, error = null, pokemon = null) }
        consulta = viewModelScope.launch {
            try {
                val pokemon = repository.obtenerPokemon(texto)
                _estado.update { it.copy(pokemon = pokemon, cargando = false) }
            } catch (e: CancellationException) {
                throw e
            } catch (e: PokemonException) {
                _estado.update { it.copy(error = e.message, cargando = false) }
            } catch (e: Exception) {
                _estado.update { it.copy(error = "No se pudo mostrar el Pokémon. Intenta nuevamente.",
cargando = false) }
            }
        }
    }
}
```

Paso 9 · La pantalla (UI)

Borra `ClimaApp.kt` y crea `PokemonApp.kt`. Reutiliza lo mismo que ya usabas: `LazyColumn`, `OutlinedTextField`, `Card` y `AsyncImage` de Coil para la imagen.

ui/PokemonApp.kt

```
package com.example.clima.ui

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalSoftwareKeyboardController
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.unit.dp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import coil.compose.AsyncImage
import com.example.clima.entidad.Pokemon
import com.example.clima.presentacion.PokemonViewModel

@Composable
fun PokemonApp(viewModel: PokemonViewModel) {
    val estado by viewModel.estado.collectAsStateWithLifecycle()
    val teclado = LocalSoftwareKeyboardController.current
    val buscar: () -> Unit = {
        teclado?.hide()
        viewModel.buscar()
    }
    Scaffold { padding ->
        LazyColumn(
            modifier = Modifier.fillMaxSize().padding(padding).imePadding(),
            contentPadding = PaddingValues(20.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            item {
                Text("Pokédex", style = MaterialTheme.typography.headlineLarge,
                    fontWeight = FontWeight.Bold)
                Text("Busca un Pokémon por nombre o número.",
                    style = MaterialTheme.typography.bodyMedium)
            }
            item {
                OutlinedTextField(
                    value = estado.busqueda, onValueChange = viewModel::cambiarBusqueda,
                    modifier = Modifier.fillMaxWidth(), singleLine = true,
                    label = { Text("Nombre o número") },
                    placeholder = { Text("ditto") },
                    keyboardOptions = KeyboardOptions(imeAction = ImeAction.Search),
                    keyboardActions = KeyboardActions(onSearch = { buscar() }),
                    shape = RoundedCornerShape(16.dp)
                )
                Spacer(Modifier.height(8.dp))
                Button(onClick = buscar, enabled = !estado.cargando,
                    modifier = Modifier.fillMaxWidth()) { Text("Buscar") }
```

```
            }
            if (estado.cargando) {
                item {
                    Column(Modifier.fillMaxWidth(), horizontalAlignment =
Alignment.CenterHorizontally) {
                        CircularProgressIndicator()
                    }
                }
            }
            estado.error?.let { mensaje ->
                item {
                    Card(colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.errorContainer)) {
                        Column(Modifier.padding(16.dp)) {
                            Text("No pudimos completar la consulta", fontWeight = FontWeight.Bold)
                            Text(mensaje)
                            TextButton(onClick = { viewModel.buscar() }) { Text("Reintentar") }
                        }
                    }
                }
            }
            estado.pokemon?.let { pokemon -> item { TarjetaPokemon(pokemon) } }
        }
    }
}

@Composable
private fun TarjetaPokemon(pokemon: Pokemon) {
    Card(shape = RoundedCornerShape(24.dp)) {
        Column(
            Modifier.fillMaxWidth().padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            AsyncImage(model = pokemon.imagen, contentDescription = pokemon.nombre,
                modifier = Modifier.size(160.dp))
            Text("#${pokemon.id} ${pokemon.nombre}",
                style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold)
            Text("Tipo: ${pokemon.tipos.joinToString(", ")}")
            Text("Altura: ${pokemon.alturaM} m  ·  Peso: ${pokemon.pesoKg} kg")
            Text("Habilidades: ${pokemon.habilidades.joinToString(", ")}")
            HorizontalDivider()
            pokemon.estadisticas.forEach { Text("${it.nombre}: ${it.valor}") }
        }
    }
}
```

Conectar la pantalla en MainActivity

MainActivity.kt

```
package com.example.clima

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.clima.presentacion.PokemonViewModel
import com.example.clima.ui.ClimaTheme
import com.example.clima.ui.PokemonApp

class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ClimaTheme {
                val pokemonViewModel: PokemonViewModel = viewModel()
                PokemonApp(pokemonViewModel)
            }
        }
    }
}
```

Paso 10 · Los tests

☐ Borra `app/src/test/resources/clima.json` y pon en esa carpeta el `ditto.json` que guardaste en el Paso 0.

☐ Borra `ClimaRepositoryTest.kt` y crea `PokemonRepositoryTest.kt` en la misma carpeta:

`test/java/com/example/clima/PokemonRepositoryTest.kt`

```
package com.example.clima

import com.example.clima.datos.*
import kotlinx.coroutines.runBlocking
import okhttp3.mockwebserver.MockResponse
import okhttp3.mockwebserver.MockWebServer
import org.junit.After
import org.junit.Assert.*
import org.junit.Before
import org.junit.Test
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class PokemonRepositoryTest {
    private lateinit var server: MockWebServer
    private lateinit var api: PokeApiService

    @Before fun preparar() {
        server = MockWebServer()
        server.start()
        api = Retrofit.Builder().baseUrl(server.url("/api/v2/"))
            .addConverterFactory(GsonConverterFactory.create()).build()
            .create(PokeApiService::class.java)
    }

    @After fun cerrar() { server.shutdown() }

    @Test fun convierteJsonYPideElPokemonCorrecto() = runBlocking {
        val json = javaClass.getResource("/ditto.json")!!.readText()
        server.enqueue(MockResponse().setBody(json).addHeader("Content-Type", "application/json"))
        val pokemon = PokemonRepository(api).obtenerPokemon("  Ditto  ")
        assertEquals(132, pokemon.id)
        assertEquals("Ditto", pokemon.nombre)
        assertEquals(0.3, pokemon.alturaM, 0.001)
        assertEquals(4.0, pokemon.pesoKg, 0.001)
        assertEquals(listOf("normal"), pokemon.tipos)
        val request = server.takeRequest(2, TimeUnit.SECONDS)!!
        assertEquals("/api/v2/pokemon/ditto", request.requestUrl!!.encodedPath)
    }

    @Test fun informaPokemonInexistente() = runBlocking {
        server.enqueue(MockResponse().setResponseCode(404).setBody("Not Found"))
        assertTrue(errorDe { PokemonRepository(api).obtenerPokemon("noexiste") }
            .contains("No encontramos"))
    }

    @Test fun rechazaBusquedaVaciaSinConsultarServidor() = runBlocking {
        assertTrue(errorDe { PokemonRepository(api).obtenerPokemon("   ") }.contains("Escribe"))
        assertEquals(0, server.requestCount)
    }
```

```
    private suspend fun errorDe(accion: suspend () -> Unit): String {
        try { accion() } catch (e: PokemonException) { return e.message.orEmpty() }
        throw AssertionError("Se esperaba PokemonException")
    }
}
```

Fíjate en que el primer test escribe `" Ditto "`, con espacios y mayúscula, y comprueba que la petición sale como `/pokemon/ditto`. Así prueba la limpieza que hace el repositorio.

Paso 11 · Compilar, probar y verificar

☐ **Build → Rebuild Project.** Si sale *Unresolved reference*, queda algún archivo viejo o un import que apunta a clases de clima. Usa **Edit → Find → Find in Files** y busca `Clima`, `WEATHER` o `RespuestaClima`.

☐ Corre los tests: clic derecho en `PokemonRepositoryTest` → **Run**.

☐ Ejecuta la app. Al abrir, debe aparecer **Ditto** con su imagen.

☐ Prueba `pikachu`, `Charizard` (con mayúscula) y `25` (número): las tres deben funcionar.

☐ Prueba `pokemonfalso`: debe salir "No encontramos ese Pokémon".

☐ Activa el modo avión: debe salir "Comprueba tu conexión a Internet".

Problemas comunes

Síntoma	Causa probable	Solución
`baseUrl must end in /`	Falta la `/` final	Paso 2
Unresolved reference: BuildConfig.WEATHER_API_KEY	Queda código que usa la clave	Busca `WEATHER_API_KEY` y bórralo
Unresolved reference: ClimaViewModel / ClimaApp	`MainActivity` sin actualizar	Paso 9
"No encontramos..." con un nombre correcto	Mayúsculas o espacios en la URL	Revisa el `lowercase()` del Paso 7
No aparece la imagen	Sin Internet o `front_default` es null	Revisa la conexión. Algunos Pokémon no tienen sprite.
"Datos que no se pudieron interpretar"	El DTO no coincide con el JSON	Compara los `@SerializedName` con `ditto.json`
El test no encuentra `/ditto.json`	El archivo está en otra carpeta	Debe estar en `app/src/test/resources/`

Opcional · Para dejarlo prolijo

☐ Cambia el nombre de la app: busca `android:label` en `AndroidManifest.xml`.

☐ Renombra `ClimaTheme` a `PokemonTheme` con clic derecho → **Refactor** → **Rename** (Shift + F6). Así Android Studio lo cambia en todos lados.

☐ El paquete `com.example.clima` puede quedarse así. Cambiarlo es más delicado: hay que tocar el `namespace` y el `applicationId` en `build.gradle.kts`.

☐ Para practicar más, muestra las estadísticas con barras (`LinearProgressIndicator`, con valor ÷ 255) o usa la imagen grande: `sprites.other["official-artwork"].front_default`.